

Katihar Engineering College, Katihar
3rd Semester(CSE) – Data Structures Laboratory

Programming Test – I

Duration: 75 Minutes

Maximum Marks: 15

General Instructions

1. Write, compile, and execute any **three** programs.
2. Use C as prescribed for the laboratory.
3. Appropriate data structures must be used for each problem.
4. Demonstrate the output using the test data given in the question.
5. Marks will be awarded for logic, implementation, testing, and appropriate handling of boundary cases.

Q1. Student Marks Management

[5 Marks]

A teacher maintains the marks of students in an array. Initially, the marks are:

72 65 81 56 90 68

Write a program that allows the teacher to:

- Add a student's marks at a specified position.
- Remove the marks of a student from a specified position.
- Search for a particular mark.
- Display the current list of marks.

Perform the following operations:

Insert 75 at position 4

Delete the element at position 2

Search for 90

Display the resulting list

Your program should appropriately handle invalid positions and unsuccessful searches.

Q2. Predictive Sequence Generator**[5 Marks]**

A monitoring system generates a sequence in which every new value is the sum of the previous two values:

0 1 1 2 3 5 8 13 ...

Implement two different approaches to generate the first N values:

- (a) Calculate each value recursively.
- (b) Store previously calculated values so that they can be reused.

For $N = 15$:

- i. Display the generated sequence using both approaches.
- ii. Compare the two approaches in terms of computation and memory.
- iii. State the time and space complexity of both approaches.
- iv. Explain how storing previously calculated values demonstrates a space-time trade-off.

Q3. Code Syntax Validator**[5 Marks]**

A compiler needs to check whether brackets in a source-code fragment are correctly nested.

For example:

```
if (a[2] > b[3]) { x = y; }
```

Write a program that checks whether:

- Every opening bracket has a corresponding closing bracket.
- Brackets are properly nested.
- The types of brackets match.

Test your program with:

```
{[()]}
```

```
{[(())]}
```

Display an appropriate message for each expression.

Q4. Simple Expression Processor**[5 Marks]**

A calculator receives the following mathematical expression:

$$A*(B+C)-D/E$$

The calculator cannot directly process the expression in the given form. Write a program to transform it into a form in which operators appear **after their operands**.

Display the transformed expression.

Your implementation should correctly handle operator precedence, associativity, and parentheses.

Q5. Calculator Engine**[5 Marks]**

A calculator internally receives an arithmetic expression in the following form:

$$62/3-42*+$$

Write a program to evaluate the expression using an appropriate data structure.

Display the intermediate results wherever an operator is processed and finally display the result.

— End of Programming Test I —